Algoritmo de Ordenação Merge Sort

Respostas dos Exercícios
Implementação em Java

Material Didático

Professor Rodrigo Bossini ⟨professorbossini@gmail.com⟩
https://professorbossini.dev

Respostas — Exercícios Teóricos

Resposta do Exercício 1 — Análise de Complexidade Linha a Linha

Considere o procedimento $\textsc{Merge}(A, p, q, r)$ aplicado à intercalação final do Merge Sort sobre o vetor $A = [38, 27, 43, 3, 9, 82, 10, 5]$ de tamanho 8. Após as chamadas recursivas, as duas metades já estão ordenadas:

$$A[1\,..\,4] = [3, 27, 38, 43] \qquad A[5\,..\,8] = [5, 9, 10, 82]$$

Os arranjos auxiliares são $L = [3, 27, 38, 43, \infty]$ e $R = [5, 9, 10, 82, \infty]$.

Resposta a)

Nesta intercalação temos $n = 8$, $n_1 = 4$, $n_2 = 4$.
Simulando passo a passo, a condição $L[i] \leq R[j]$ é verdadeira nos passos $k = 1$ $(3 \leq 5)$, $k = 5$ $(27 \leq 82)$, $k = 6$ $(38 \leq 82)$ e $k = 7$ $(43 \leq 82)$. Portanto $\tau = 4$ e $n - \tau = 4$.

Linha	Trecho	Custo	Execuções
1	$n_1 \leftarrow q - p + 1$	c_1	1
2	$n_2 \leftarrow r - q$	c_2	1
3	criar L, R	c_3	1
4	**para** $i \leftarrow 1$ **até** n_1 (teste)	c_4	$4 + 1 = 5$
5	$\quad L[i] \leftarrow A[p + i - 1]$	c_5	4
6	**para** $j \leftarrow 1$ **até** n_2 (teste)	c_6	$4 + 1 = 5$
7	$\quad R[j] \leftarrow A[q + j]$	c_7	4
8	$L[n_1{+}1] \leftarrow \infty$	c_8	1
9	$R[n_2{+}1] \leftarrow \infty$	c_9	1
10	$i \leftarrow 1;\ j \leftarrow 1$	c_{10}	1
12	**para** $k \leftarrow p$ **até** r (teste)	c_{12}	$8 + 1 = 9$
13	$\quad$ **se** $L[i] \leq R[j]$	c_{13}	8
14	$\quad\quad A[k] \leftarrow L[i];\ i \leftarrow i + 1$	c_{14}	$\tau = 4$
16	$\quad\quad A[k] \leftarrow R[j];\ j \leftarrow j + 1$	c_{16}	$n - \tau = 4$

Resposta b)

$$T_{\text{Merge}}(8) = c_1 + c_2 + c_3 + 5c_4 + 4c_5 + 5c_6 + 4c_7 + c_8 + c_9 + c_{10} + 9c_{12} + 8c_{13} + 4c_{14} + 4c_{16}$$

De forma geral, $n_1 + n_2 = n$ e todos os termos dominantes são proporcionais a n. Agrupando:

$$T_{\text{Merge}}(n) = \underbrace{(c_4 + c_5)n_1 + (c_6 + c_7)n_2}_{\text{cópias para } L,R} + \underbrace{(c_{12} + c_{13})n + (c_{14}\tau + c_{16}(n-\tau))}_{\text{intercalação}} + \underbrace{\text{constantes}}_{\Theta(1)}$$

Como $n_1+n_2 = n$ e $\tau + (n-\tau) = n$, todos os termos são $an + b$, portanto:

$$\boxed{T_{\text{Merge}}(n) = \Theta(n)}$$

Resposta c)

No Insertion Sort, o custo depende de t_j (quantas vezes o laço **while** executa para cada j), que varia conforme a ordem da entrada: de $t_j = 1$ (melhor caso) até $t_j = j$ (pior caso).
No MERGE, o laço principal **para** $k \leftarrow p$ **até** r sempre executa exatamente n iterações, independentemente da ordem dos elementos. O valor de τ muda (quantas vezes copia de L vs. de R), mas a soma $\tau + (n - \tau) = n$ é constante. Por isso, o MERGE é sempre $\Theta(n)$ — não possui distinção entre melhor e pior caso.

Resposta do Exercício 2 — Recorrência e Árvore de Recorrência

Considere a recorrência do Merge Sort:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ 2\,T(n/2) + \Theta(n) & \text{se } n > 1 \end{cases}$$

Resposta a)

Para $n = 16$ ($\lg 16 = 4$), a árvore tem $\lg n + 1 = 5$ níveis:

Nível	Nós	Tamanho de cada	Custo total
0	1 nó	$cn = 16c$	$16c$
1	2 nós	$cn/2 = 8c$	$2 \times 8c = 16c$
2	4 nós	$cn/4 = 4c$	$4 \times 4c = 16c$
3	8 nós	$cn/8 = 2c$	$8 \times 2c = 16c$
4	16 nós	c	$16 \times c = 16c$

Cada nível custa $cn = 16c$. São $\lg 16 + 1 = 5$ níveis.

Resposta b)

A árvore possui $\lg n + 1$ níveis, cada um com custo cn. Somando:

$$T(n) = cn \cdot (\lg n + 1) = cn \lg n + cn$$

Para $n = 16$: $T(16) = 16c \cdot 4 + 16c = 64c + 16c = 80c$.

Resposta c)

Cota superior. Para $n \geq 2$, temos $\lg n \geq 1$, logo $n \leq n \lg n$:

$$cn \lg n + cn \leq cn \lg n + cn \lg n = 2c\, n \lg n$$

Escolhendo $c_2 = 2c$ e $n_0 = 2$.
Cota inferior. Como $cn > 0$:

$$cn \lg n + cn \geq cn \lg n$$

Escolhendo $c_1 = c$ e $n_0 = 2$.
Portanto, para todo $n \geq 2$:

$$c\, n \lg n \;\leq\; cn \lg n + cn \;\leq\; 2c\, n \lg n$$

$$\boxed{T(n) = cn \lg n + cn = \Theta(n \lg n)}$$

Respostas — Exercícios Práticos

Resposta do Exercício 3 — Implementação com Contagem de Operações

Código 1: MergeSortContador.java

```java
import java.util.Arrays;
import java.util.Random;
import java.util.Scanner;

public class MergeSortContador {

    static int comparacoes = 0;
    static int copias = 0;

    public static void mergeSort(int[] A, int p, int r) {
        if (p < r) {
            int q = (p + r) / 2;
            mergeSort(A, p, q);
            mergeSort(A, q + 1, r);
            merge(A, p, q, r);
        }
    }

    public static void merge(int[] A, int p, int q, int r) {
        int n1 = q - p + 1;
        int n2 = r - q;

        int[] L = new int[n1 + 1];
        int[] R = new int[n2 + 1];

        for (int i = 0; i < n1; i++) {
            L[i] = A[p + i];
        }
        for (int j = 0; j < n2; j++) {
            R[j] = A[q + 1 + j];
        }

        L[n1] = Integer.MAX_VALUE;
        R[n2] = Integer.MAX_VALUE;

        int i = 0, j = 0;
        for (int k = p; k <= r; k++) {
            comparacoes++;
            if (L[i] <= R[j]) {
                A[k] = L[i];
                i++;
            } else {
                A[k] = R[j];
```

```
                    j++;
                }
                copias++;
            }
        }

        public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);
            System.out.print("Digite n: ");
            int n = sc.nextInt();

            Random rand = new Random();
            int[] A = new int[n];
            for (int i = 0; i < n; i++) {
                A[i] = rand.nextInt(1000) + 1;
            }

            System.out.println("Antes: "
                + Arrays.toString(A));

            comparacoes = 0;
            copias = 0;
            mergeSort(A, 0, A.length - 1);

            System.out.println("Depois: "
                + Arrays.toString(A));
            System.out.println("Comparacoes: "
                + comparacoes);
            System.out.println("Copias: " + copias);
        }
    }
```

Análise dos cenários para n

Cenário	Comparações	Cópias	Complexidade
Já ordenado (crescente)	17	24	$\Theta(n \lg n)$
Ordem decrescente	17	24	$\Theta(n \lg n)$
Aleatório	≈ 17	24	$\Theta(n \lg n)$

O número de comparações **não muda significativamente** entre os cenários. Isso ocorre porque o laço de intercalação do MERGE sempre executa exatamente n iterações por chamada, independentemente da ordem dos elementos. O número de cópias é também sempre n por chamada de MERGE. Como a estrutura recursiva é a mesma (sempre divide ao meio), o total de operações é $\Theta(n \lg n)$ em qualquer caso.
Isso contrasta fortemente com o Insertion Sort, onde o vetor ordenado gera $\Theta(n)$ e o decrescente gera $\Theta(n^2)$.

Resposta do Exercício 4 — Merge Sort com Contagem de Inversões

Código 2: ContadorInversoes.java

```java
import java.util.Arrays;
import java.util.Random;
import java.util.Scanner;

public class ContadorInversoes {

    static long inversoes = 0;

    public static void mergeSortConta(int[] A,
            int p, int r) {
        if (p < r) {
            int q = (p + r) / 2;
            mergeSortConta(A, p, q);
            mergeSortConta(A, q + 1, r);
            mergeConta(A, p, q, r);
        }
    }

    public static void mergeConta(int[] A,
            int p, int q, int r) {
        int n1 = q - p + 1;
        int n2 = r - q;

        int[] L = new int[n1];
        int[] R = new int[n2];

        for (int i = 0; i < n1; i++) {
            L[i] = A[p + i];
        }
        for (int j = 0; j < n2; j++) {
            R[j] = A[q + 1 + j];
        }

        int i = 0, j = 0, k = p;
        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) {
                A[k] = L[i];
                i++;
            } else {
                A[k] = R[j];
                // Todos os elementos restantes
                // de L formam inversao com R[j]
                inversoes += (n1 - i);
                j++;
            }
            k++;
        }

        while (i < n1) {
```

```java
                A[k] = L[i];
                i++;
                k++;
            }
            while (j < n2) {
                A[k] = R[j];
                j++;
                k++;
            }
        }

        public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);
            System.out.print("Digite n: ");
            int n = sc.nextInt();

            Random rand = new Random();
            int[] A = new int[n];
            for (int i = 0; i < n; i++) {
                A[i] = rand.nextInt(100) + 1;
            }

            System.out.println("Vetor original: "
                + Arrays.toString(A));

            inversoes = 0;
            mergeSortConta(A, 0, A.length - 1);

            System.out.println("Vetor ordenado: "
                + Arrays.toString(A));
            System.out.println("Numero de inversoes: "
                + inversoes);
        }
    }
```

Análise de complexidade

A solução tem complexidade $\Theta(n \lg n)$, a mesma do Merge Sort padrão. A única modificação é a linha `inversoes += (n1 - i)`, que é uma operação $\Theta(1)$ executada dentro do laço de intercalação, sem alterar a complexidade.

A abordagem ingênua de verificar todos os pares (i, j) com $i < j$ requer dois laços aninhados: $\binom{n}{2} = \frac{n(n-1)}{2} = \Theta(n^2)$ comparações.

A adaptação do Merge Sort é, portanto, assintoticamente muito mais eficiente: $\Theta(n \lg n)$ contra $\Theta(n^2)$.

Verificação para $A = [5, 3, 8, 1, 2]$: as inversões são $(5, 3)$, $(5, 1)$, $(5, 2)$, $(3, 1)$, $(3, 2)$, $(8, 1)$, $(8, 2)$ — total de **7** inversões.

Referências

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Introduction to Algorithms*. 4.ª edição. MIT Press, 2022. Capítulo 2: Getting Started; Capítulo 4: Divide-and-Conquer.